Departamento de Electrónica, Sistemas e Informática

Big Data

Spring 2026

Examples on Structured Streaming (Kakfa producer)

Profesor: Pablo Camarillo Ramirez

Alumno: Bryan Edgardo Romo Gonzalez

Create SparkSession

In [1]:
```
from spark_utils import SparkUtils
from pathlib import Path
import shutil
import pyspark.sql.functions as F

kafka_connector = "org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.0"
su = SparkUtils("Example Kafka",
                "spark://spark-master:7077",
```

```
                spark_packages=kafka_connector)
su._spark
```

```
WARNING: Using incubator modules: jdk.incubator.vector
:: loading settings :: url = jar:file:/opt/spark/jars/ivy-2.5.3.jar!/org/apache/ivy/core/settings/ivysettings.xml
Ivy Default Cache set to: /root/.ivy2.5.2/cache
The jars for the packages stored in: /root/.ivy2.5.2/jars
org.apache.spark#spark-sql-kafka-0-10_2.13 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-9d97d57a-b54e-483a-b386-c09c7314ead1;1.0
        confs: [default]
        found org.apache.spark#spark-sql-kafka-0-10_2.13;4.0.0 in central
        found org.apache.spark#spark-token-provider-kafka-0-10_2.13;4.0.0 in central
        found org.apache.kafka#kafka-clients;3.9.0 in central
        found org.lz4#lz4-java;1.8.0 in central
        found org.xerial.snappy#snappy-java;1.1.10.7 in central
        found org.slf4j#slf4j-api;2.0.16 in central
        found org.apache.hadoop#hadoop-client-runtime;3.4.1 in central
        found org.apache.hadoop#hadoop-client-api;3.4.1 in central
        found com.google.code.findbugs#jsr305;3.0.0 in central
        found org.scala-lang.modules#scala-parallel-collections_2.13;1.2.0 in central
        found org.apache.commons#commons-pool2;2.12.0 in central
:: resolution report :: resolve 1084ms :: artifacts dl 41ms
        :: modules in use:
        com.google.code.findbugs#jsr305;3.0.0 from central in [default]
        org.apache.commons#commons-pool2;2.12.0 from central in [default]
        org.apache.hadoop#hadoop-client-api;3.4.1 from central in [default]
        org.apache.hadoop#hadoop-client-runtime;3.4.1 from central in [default]
        org.apache.kafka#kafka-clients;3.9.0 from central in [default]
        org.apache.spark#spark-sql-kafka-0-10_2.13;4.0.0 from central in [default]
        org.apache.spark#spark-token-provider-kafka-0-10_2.13;4.0.0 from central in [default]
        org.lz4#lz4-java;1.8.0 from central in [default]
        org.scala-lang.modules#scala-parallel-collections_2.13;1.2.0 from central in [default]
        org.slf4j#slf4j-api;2.0.16 from central in [default]
        org.xerial.snappy#snappy-java;1.1.10.7 from central in [default]
        ---------------------------------------------------------------------
        |                  |            modules            ||   artifacts   |
        |       conf       | number| search|dwnlded|evicted|| number|dwnlded|
        ---------------------------------------------------------------------
        |      default     |   11  |   0   |   0   |   0   ||   11  |   0   |
        ---------------------------------------------------------------------
:: retrieving :: org.apache.spark#spark-submit-parent-9d97d57a-b54e-483a-b386-c09c7314ead1
        confs: [default]
        0 artifacts copied, 11 already retrieved (0kB/29ms)
```

```
26/04/14 01:17:09 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes w
here applicable
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
```

Out[1]:

SparkSession - in-memory

SparkContext

Spark UI

Version	`v4.0.1`
Master	`spark://spark-master:7077`
AppName	`Example Kafka`

Create a data stream from a Kafka topic

In [ ]:

```
# Create the remote connection
kafka_df = su.spark.readStream \
            .format("kafka") \
            .option("kafka.bootstrap.servers", "kafka:9093") \
            .option("subscribe", "kafka-spark-example") \
            .load()

kafka_df.printSchema()

# Transform binary data to string
df_input = kafka_df.selectExpr("CAST(value AS STRING)")

# Clean checkpoint
checkpoint_path = "/opt/spark/work-dir/checkpoints/logs_checkpoint"
dir_path = Path(checkpoint_path)
if dir_path.exists() and dir_path.is_dir():
    shutil.rmtree(dir_path)

words = df_input.select(F.explode(F.split(df_input.value, " ")).alias("word"))
word_count = words.groupBy("word").count()
```

```
# Send transformed data to the Sink
query_a = (word_count.writeStream
            .trigger(processingTime='2 second')
            .outputMode("complete")
            .format("console")
            .option("checkpointLocation", checkpoint_path)
            .start())

print("   Press Ctrl+C to stop.\n")
su.spark.streams.awaitAnyTermination()
```

```
root
 |-- key: binary (nullable = true)
 |-- value: binary (nullable = true)
 |-- topic: string (nullable = true)
 |-- partition: integer (nullable = true)
 |-- offset: long (nullable = true)
 |-- timestamp: timestamp (nullable = true)
 |-- timestampType: integer (nullable = true)
```

```
26/04/14 00:41:44 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and w
ill be disabled.
   Press Ctrl+C to stop.
```

```
-------------------------------------------
Batch: 0
-------------------------------------------
+----+-----+
|word|count|
+----+-----+
+----+-----+
```

```
26/04/14 00:41:51 WARN ProcessingTimeExecutor: Current batch is falling behind. The trigger interval is 2000} milliseconds, but
spent 6761 milliseconds
26/04/14 00:53:07 ERROR TaskSchedulerImpl: Lost executor 0 on 172.18.0.5: worker lost: Not receiving heartbeat for 60 seconds
26/04/14 01:01:26 WARN HeartbeatReceiver: Removing executor 1 with no recent heartbeats: 202508 ms exceeds timeout 120000 ms
26/04/14 01:01:26 ERROR TaskSchedulerImpl: Lost executor 1 on 172.18.0.5: Executor heartbeat timed out after 202508 ms
```

Custom producer

Create `server-logs` topic

```
docker exec -it 638e00fabc45 /opt/kafka/bin/kafka-topics.sh --create --zookeeper zookeeper:2181 --
replication-factor 1 --partitions 1 --topic server-logs
```

Run the producer

```
docker exec -it <Spark-Notebook container ID> /bin/bash
# cd src/producers/
# python3 kafka_producer.py --broker kafka:9093 --topic server-logs --records 20
```

Run the consumer code

In [2]:
```
# Create the remote connection
server_logs_df = (su.spark.readStream
            .format("kafka")
            .option("kafka.bootstrap.servers", "kafka:9093")
            .option("subscribe", "server-logs")
            .load())

# Transform binary data to string
logs_df = server_logs_df.selectExpr("CAST(value AS STRING)")

# Clean checkpoint
checkpoint_path = "/opt/spark/work-dir/checkpoints/logs_checkpoint"
dir_path = Path(checkpoint_path)
if dir_path.exists() and dir_path.is_dir():
    shutil.rmtree(dir_path)

# Transform original dataframe
parsed_df = (
```

```
    logs_df
    .withColumn("parts",     F.split(F.col("value"), r" \| "))
    .withColumn("timestamp", F.to_timestamp(F.col("parts")[0], "yyyy-MM-dd HH:mm:ss"))
    .withColumn("level",     F.trim(F.col("parts")[1]))
    .withColumn("message",   F.trim(F.col("parts")[2]))
    .withColumn("server",    F.trim(F.col("parts")[3]))
    .drop("parts", "value")
    .filter(F.col("timestamp").isNotNull())
)

# Write stream in the destination
output_path = "/opt/spark/work-dir/data/streaming/output/"
query_events = (
    parsed_df.writeStream
    .outputMode("append")
    .format("parquet")
    .option("truncate", False)
    .option("checkpointLocation", checkpoint_path)
    .option("path", output_path)
    .partitionBy("server", "level")
    .start()
)

print("   Press Ctrl+C to stop.\n")
su.spark.streams.awaitAnyTermination()
```

```
26/04/14 01:17:41 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and w
ill be disabled.
   Press Ctrl+C to stop.
```

```
ERROR:root:KeyboardInterrupt while sending command.
Traceback (most recent call last):
  File "/opt/spark/python/lib/py4j-0.10.9.9-src.zip/py4j/java_gateway.py", line 1038, in send_command
    response = connection.send_command(command)
  File "/opt/spark/python/lib/py4j-0.10.9.9-src.zip/py4j/clientserver.py", line 535, in send_command
    answer = smart_decode(self.stream.readline()[:-1])
  File "/usr/lib/python3.10/socket.py", line 705, in readinto
    return self._sock.recv_into(b)
KeyboardInterrupt
ERROR:py4j.clientserver:Exception occurred while shutting down connection
Traceback (most recent call last):
  File "/opt/spark/python/lib/py4j-0.10.9.9-src.zip/py4j/java_gateway.py", line 1038, in send_command
    response = connection.send_command(command)
  File "/opt/spark/python/lib/py4j-0.10.9.9-src.zip/py4j/clientserver.py", line 535, in send_command
    answer = smart_decode(self.stream.readline()[:-1])
  File "/usr/lib/python3.10/socket.py", line 705, in readinto
    return self._sock.recv_into(b)
KeyboardInterrupt

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "/opt/spark/python/lib/py4j-0.10.9.9-src.zip/py4j/clientserver.py", line 504, in shutdown_socket
    self.socket.sendall(("%s\n" % address).encode("utf-8"))
BrokenPipeError: [Errno 32] Broken pipe
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[2], line 43
     31 query_events = (
     32     parsed_df.writeStream
     33     .outputMode("append")
   (...)
     39     .start()
     40 )
     42 print("   Press Ctrl+C to stop.\n")
---> 43 su.spark.streams.awaitAnyTermination()

File /opt/spark/python/pyspark/sql/streaming/query.py:600, in StreamingQueryManager.awaitAnyTermination(self, timeout)
    598     return self._jsqm.awaitAnyTermination(int(timeout * 1000))
    599 else:
--> 600     return self._jsqm.awaitAnyTermination()

File /opt/spark/python/lib/py4j-0.10.9.9-src.zip/py4j/java_gateway.py:1361, in JavaMember.__call__(self, *args)
   1354 args_command, temp_args = self._build_args(*args)
   1356 command = proto.CALL_COMMAND_NAME +\
   1357     self.command_header +\
   1358     args_command +\
   1359     proto.END_COMMAND_PART
-> 1361 answer = self.gateway_client.send_command(command)
   1362 return_value = get_return_value(
   1363     answer, self.gateway_client, self.target_id, self.name)
   1365 for temp_arg in temp_args:

File /opt/spark/python/lib/py4j-0.10.9.9-src.zip/py4j/java_gateway.py:1038, in GatewayClient.send_command(self, command, retry,
binary)
   1036 connection = self._get_connection()
   1037 try:
-> 1038     response = connection.send_command(command)
   1039     if binary:
   1040         return response, self._create_connection_guard(connection)

File /opt/spark/python/lib/py4j-0.10.9.9-src.zip/py4j/clientserver.py:535, in ClientServerConnection.send_command(self, comman
d)
    533 try:
    534     while True:
```

```
--> 535         answer = smart_decode(self.stream.readline()[:-1])
    536         logger.debug("Answer received: {0}".format(answer))
    537         # Happens when a the other end is dead. There might be an empty
    538         # answer before the socket raises an error.

File /usr/lib/python3.10/socket.py:705, in SocketIO.readinto(self, b)
    703 while True:
    704     try:
--> 705         return self._sock.recv_into(b)
    706     except timeout:
    707         self._timeout_occurred = True

KeyboardInterrupt:
```

In [3]:
```
!ls -lah /opt/spark/work-dir/data/streaming/output/
```

```
total 0
drwxr-xr-x 1 root root 4.0K Apr 14 01:08  .
drwxrwxrwx 1 root root 4.0K Apr 14 01:05  ..
drwxr-xr-x 1 root root 4.0K Apr 14 01:08 'server=server-node-1'
drwxr-xr-x 1 root root 4.0K Apr 14 01:08 'server=server-node-2'
drwxr-xr-x 1 root root 4.0K Apr 14 01:06 'server=server-node-3'
drwxr-xr-x 1 root root 4.0K Apr 14 01:08 'server=server-node-4'
drwxr-xr-x 1 root root 4.0K Apr 14 01:06 'server=server-node-5'
drwxr-xr-x 1 root root 4.0K Apr 14 01:09  _spark_metadata
```

In [ ]:
```
su.spark.stop()
```